Université de Monastir
Faculté des Sciences de Monastir
Département Informatique

Cours Intitulé :

Programmation Mobile Android

Présenté par : Soufien GDAIM

PLAN

- 1. Introduction aux Systèmes d'exploitation Mobiles (Android)
- 2. Environnement de Développement
- 3. Cycle de Vie d'une Activité
- 4. Interfaces graphiques
- 5. Les Intents
- 6. Persistance des données
- 7. Capteurs sous Android
- 8. Programmation Concurrente

Systèmes d'exploitation Mobiles

- Un système d'exploitation mobile est conçu spécialement pour les systèmes mobiles : Smartphones, PDA, Tablette, Notebook, minipc, Tablette voitures, réveil et terminaux mobiles
- Il offre :
  - Gestion d'alimentation : batterie
  - Gestion mémoire : RAM
  - Gestion sans fil : wifi, bluetooth, 3G, 4G, …
  - Gestion des connectivités :USB,HDMI, …
  - Gestion des supports de stockages : SD-CARD
  - Exécution de plusieurs applications,



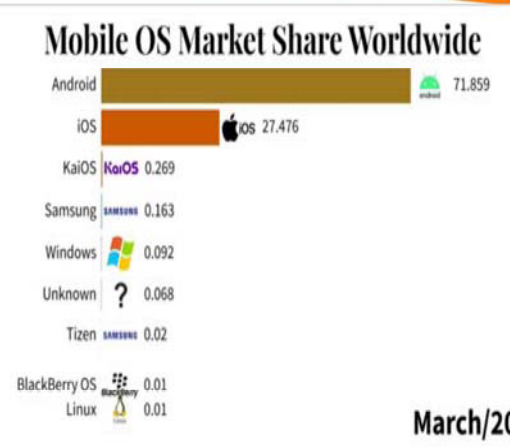




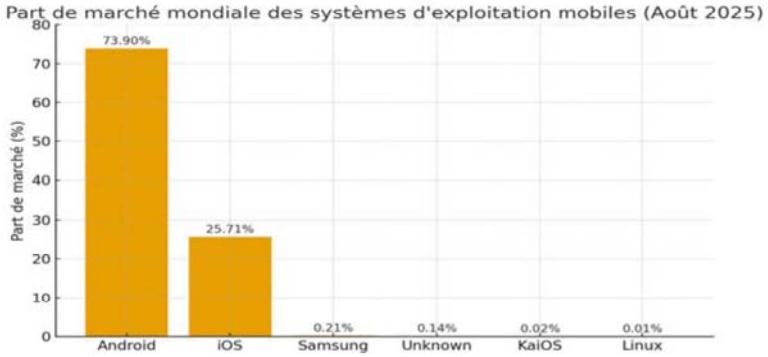
Systèmes d'exploitation Mobiles

- Les Principaux concurrents
  - Android
  - IOS (iphone, ipad)
  - Window Mobile (Microsoft)
  - Symbian OS (Nokia)
  - Web OS (Palm)
  - BlackBerry OS (RIM)
  - Bada (Samsung) (même si Samsung utilise aussi Android)

https://www.youtube.com/watch?v=Rt6G8DNHAsI



Systèmes d'exploitation Mobiles



https://gs.statcounter.com/os-market-share/mobile/worldwide

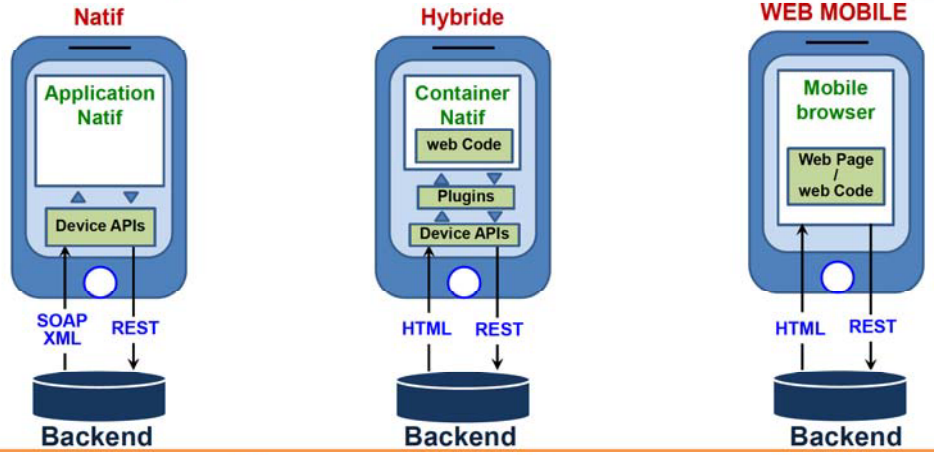
Choix d'un Système d'exploitation Mobile

- Problème de choix d'un OS mobile cible :



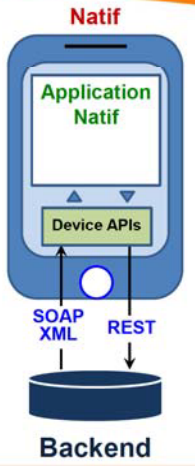
Développement des applications Mobiles

- Les différents types d'applications:



Développement Natif ou Hybride ou Web Mobile

- Le développement Natif :
  - Une application mobile native est développée spécifiquement pour une plateforme précise, cela veut dire qu'une application mobile iPhone ne fonctionne pas sur Android et vise vers ça.
  - Chaque plateforme mobile à ses propres outils de développements et ses propres langages de programmations.
  - Exploitation maximale des fonctionnalités du dispositif mobile



Développement Natif ou Hybride ou Web Mobile

- Le développement Natif :

BUILD PROGRAM	Java kotlin	Objective-C Swift	C# .Net
DEPLOY SOLUTIONS			

Développement Natif ou Hybride ou Web Mobile

- Le développement Natif :

Technologies / Frameworks

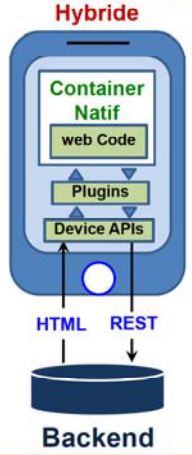
Client	Serveur
Kotlin, Objective-C, Java, Microsoft .NET	python, Flask, django, php, Laravel, Symfony, node, JavaEE, spring, spring boot, HIBERNATE JAVA ORM

Développement Natif ou Hybride ou Web Mobile

- Le développement Natif :
  - Avantages :
    - Permet de développer avec design respectant chaque plateforme.
    - Accès total et performant au ressources matériels smartphone (caméra, appareil photo, microphone, géolocalisation, le stockage, l'agenda, les contacts…).
    - Code générer plus rapide, plus sécurisé, plus stable et plus optimisé
    - Permet d'accéder facilement au dernières nouveautés des SDK
  - Inconvénients :
    - Nécessite de multiples compétences
    - Nécessite autant de développements que de plateformes cibles
    - Maintenance plus compliquée
    - Nécessite de multiples environnement

Développement Natif ou Hybride ou Web Mobile

- Le développement Hybride :
  - Une application hybride est composée de 80% du web (HTML/CSS/JS) et de 20% d'extensions enveloppée d'une surcouche native adaptée aux différents systèmes d'exploitation et donc elle permet l'accès à certaines fonctionnalités natives.
  - Les applications développées en hybride sont déployées sur plusieurs plateformes
  - Maintenance simple
  - Coût et temps de développement réduit
  - Interface utilisateur limitée et non fluide.



Développement Natif ou Hybride ou Web Mobile

- Le développement Hybride :

BUILD PROGRAM

Single Page Application + Codenatif (plugins)

DEPLOY SOLUTIONS

+ iOS +

Développement Natif ou Hybride ou Web Mobile

- Le développement Hybride :



Package Natif



Code Natif

Développement Natif ou Hybride ou Web Mobile

- Le développement Hybride :

Technologies / Framework

Client	Serveur
HTML, JS, CSS, Bootstrap, Mobile Angular UI, Java, Flutter, React Native, .NET, Objective-C	python, Flask, django, php, Laravel, Symfony, node, JavaEE, spring, spring boot, HIBERNATE JAVA ORM

Développement Natif ou Hybride ou Web Mobile

- Le développement Hybride :
  - Le résultat d'une application hybride est une application packagée pour chaque plateforme.
  - Cela impose d'avoir à disposition : du matériel et un système pour chaque plateforme cible.

SDK Android
Android Studio

iOS
Mac OS
Xcode

SDK .Net
Visual Studio

Développement Natif ou Hybride ou Web Mobile

- **Le développement Hybride :**
  - **Avantages :**
    - Technologies traditionnelles du web donc compétences largement disponibles
    - Possibilité d'étendre le container natif par un système de plugin natif permettant d'accéder a toutes les APIs natives,
    - Simplicité du développement (code unique implémenté pour plusieurs plateformes iOS, Android …).
    - Développement rapide que le développement natif
  - **Inconvénients :**
    - Accessibilité partielle aux fonctionnalités natives
    - Interface utilisateur non fluide et non optimisée
    - Impossible de respecter parfaitement le design spécifique de chaque plateforme
    - Performances et fonctionnalités très limitées

Développement Natif ou Hybride ou Web Mobile

- **Le développement Web Mobile :**
  - Une application Web Mobile est une application mobile développée avec des langages de développement web (HTML5, CSS3, js …) et exécutée depuis un navigateur web mobile.
  - Un navigateur est essentiel pour leurs fonctionnement
  - Il n'y a pas besoin d'installer une application et les mises à jour sont transparentes pour l'utilisateur.
  - Elle ne peut pas fonctionner sans connexion Internet.
  - L'exploitation des fonctionnalités des dispositifs mobiles est réduite



Développement Natif ou Hybride ou Web Mobile

- **Le développement Web Mobile :**

BUILD PROGRAM

Technologie web server side (jsp …)

Single Page Application

DEPLOY SOLUTIONS

+ iOS +

Web Browsers

Développement Natif ou Hybride ou Web Mobile

- **Le développement Web Mobile :**

Technologies / Frameworks

Client

Bootstrap

ANGULARJS

jQuery

Serveur

python Flask django

php Laravel Symfony

node

JavaEE spring spring boot HIBERNATE JAVA ORM

Développement Natif ou Hybride ou Web Mobile

- **Le développement Web Mobile :**
  - **Avantages :**
    - Technologies traditionnelles du web, donc compétences largement disponibles
    - L'avantage majeur d'une application web c'est qu'elle coûte moins chère qu'une application développée en natif.
    - Le temps de développement de ces applications est le plus rapide
    - Directement accessible dans un navigateur (pas de store)
  - **Inconvénients :**
    - Accès limité au matériel
    - Impossible de respecter parfaitement le design spécifique de chaque plateforme
    - Ne peut pas envoyer des notifications push
  - **Contraintes / bonnes pratiques :**
    - Implémentation Responsive Design / Adaptative Design

Développement Natif ou Hybride ou Web Mobile

- **Synthèse comparative:**

	Natif	Hybride	Web Mobile
Temps de développement			
Performance			
Cout de développement			
Cout du multiplateforme			
Compétences disponibles			
Distribution via App Store, Play Store			
Accès au matériel / au système			
Accès aux nouveautés des plateformes			
Intégration dans une boite de développement			

Développement Natif ou Hybride ou Web Mobile

- **Le meilleur choix?**
  - Il n'y a pas de meilleur choix, le choix dépend clairement du contexte.
  - La question à se poser :
    - Application grand public?
    - Application de gestion interne?
    - La performance est-il un critère important?
  - Choisir entre une application mobile native, hybride ou web ce n'est pas du tout facile. Votre choix dépend de vos besoins, vos objectifs, votre budget et de vos utilisateurs.
    - pour créer une application de jeu il vous faut une application native puisqu'elle demande des performances,
    - pour développer un seul code pour des différents dispositifs, gagner du temps et budget, il faut que vous choisissiez le développement hybride,
    - pour créer une application de besoin fonctionnel simple et accessible par toutes les plateformes il faut une application web.

Systèmes d'exploitation Mobiles

- Dans ce cours, on s'intéressera au système d'exploitation mobile Android et on va choisir le type de développement Natif pour développer les applications.





Systèmes d'exploitation Mobiles : Android



- Un système d'exploitation open source pour Smartphones, PDA, Tablette, Notebook, minipc, Tablette voitures, réveil et terminaux mobiles
- Conçu par Android, une startup rachetée par Google, et annoncé officiellement le 5 novembre 2007
- Le symbole d'Android est un petit robot vert



Historique d'Android

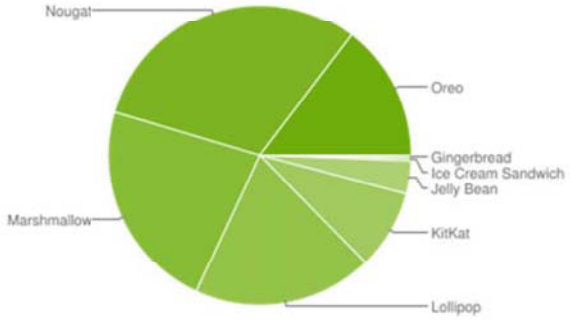
https://socialcompare.com/fr/comparison/android-versions-comparison

	Name	Version number(s)	Initial stable, release date	API level
Android 1	Apple pie	1.0	September 23, 2008	1
Android 1	Banana bread	1.1	February 9, 2009	2
Android 1	Cupcake	1.5	April 27, 2009	3
Android 1	Donut	1.6	September 15, 2009	4
Android 2	Éclair	2.0 – 2.1	October 26, 2009	5 – 7
Android 2	Froyo	2.2 – 2.2.3	May 20, 2010	8
Android 2	Gingerbread	2.3 – 2.3.7	December 6, 2010	9 – 10
Android 3	Honeycomb	3.0 – 3.2.6	February 22, 2011	11 – 13
Android 4	Ice Cream Sandwich	4.0 – 4.0.4	October 18, 2011	14 – 15
Android 4	Jelly Bean	4.1 – 4.3.1	July 9, 2012	16 – 18
Android 4	KitKat	4.4 – 4.4.4	October 31, 2013	19 – 20
Android 5	Lollipop	5.0 – 5.1.1	November 12, 2014	21 – 22
Android 6	Marshmallow	6.0 – 6.0.1	October 5, 2015	23
Android 7	Nougat	7.0 – 7.1.2	August 22, 2016	24 – 25
Android 8	Oreo	8.0 – 8.1	August 21, 2017	26 – 27
Android 9	Pie	9	August 6, 2018	28
Android 10	Queen Cake	10	September 3, 2019	29
Android 11	Red Velvet Cake	11	September 8, 2020	30
Android 12	Snow Cone/Snow Cone V2	12 / 12L	Octobre 2021	31/32
Android 13	Tiramisu	13	August 2022	33
Android 14	Upside Down Cake	14	August 2023	34
Android 15	Vanilla Ice Cream	15	2024	

Les versions d'Android

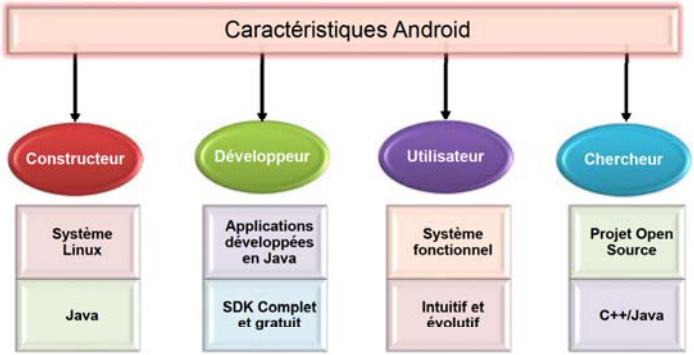


Les versions d'Android les plus populaires (2018-2019)

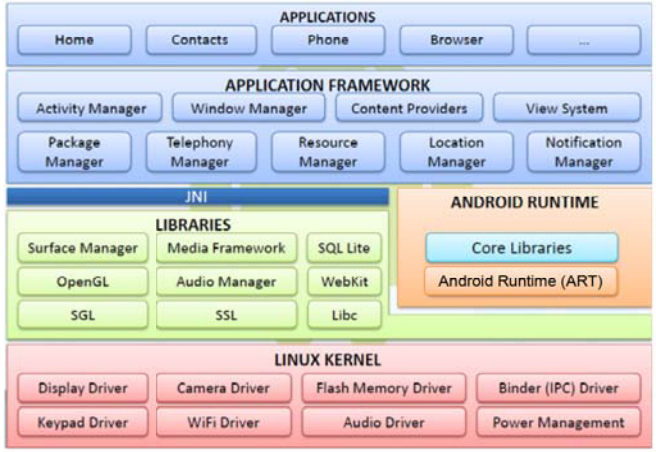


Google Play Store septembre 2018

Caractéristiques



Architecture Logicielle : Linux Inside



Architecture Logicielle

- Linux kernel (version 3.4 pour Lollipop 5.1)
  - Offre les services fondamentaux : hardware drivers, process & memory management, security, network, power management.
  - Offre une couche d'abstraction entre le HW et le reste de la pile logicielle.

    - Architecture ARM (famille RISC)
    - Système de fichiers supporté : FAT32
    - Support de TCP/IP, UDP



Architecture Logicielle

- Surface Manager (écran)
- SQL Lite, OpenGL (Graphique 2D/3D)
- Audio Manager (sonneries, écouteurs)



Android Runtime (ART)
(JVM pour PC)

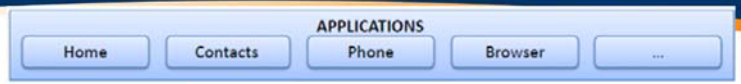
Architecture Logicielle

- Framework (pour développeurs)



- Fournit les classes qui seront utilisées pour la création d'applications Android.
  - Activity Manager (Intents, Actions)
  - Location Manager (GPS, Coordonnées Map)
  - Notification Manager (Barre de notification, Vibreur, Son, LED)
  - Telephony Manager (Contacts, SMS …)

Architecture Logicielle



- Composants d'une Application:
  - Activity
  - Service
  - Intent
  - Fragment
  - Content Provider
  - Broadcast Receiver

Architecture Logicielle: Activity

- Activity (Activité)
  - Correspond à la partie présentation de l'application,
  - Fonctionne par le biais de vues qui affichent des interfaces graphiques et répondent aux actions utilisateur.
  - Une activité correspond à un écran mais elle peut contenir plusieurs vues.



- Classe + Interface Graphique (un écran)
- Répond à des évènements (click, touche,…)
- Cycle de Vie (Création → Destruction)
- 1 Activité au moins par Application

Architecture Logicielle: Service

- Le service est destiné à fonctionner indépendamment des activités.
  - Processus
  - Sans Interface Graphique
  - Fonctionne en Background
  - Interne / Externe
  - Ses principales utilisations sont
    - la mise à jour de sources de données
    - la mise à jour des activités visibles
    - Le déclenchement de notifications.
- classe `Service`

Architecture Logicielle: Content Provider

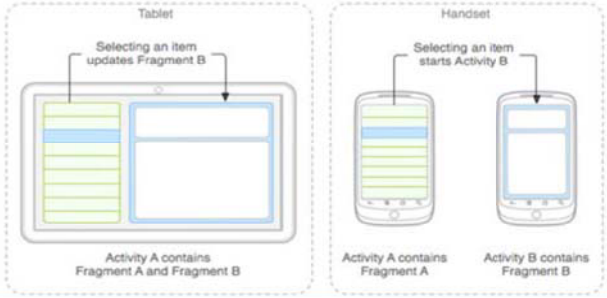
- **Content provider (fournisseurs de contenus) permet de gérer et de partager des informations.**
- **Un même fournisseur permet d'accéder à des données au sein d'une application et entre applications (partage des données).**
  - Partage de données entre application
  - Liste de Contacts
  - Agenda
  - Photos
  - SQLite
  - GPS, Module GSM, UMTS
- **classe `ContentProvider`**

Architecture Logicielle: Broadcast Receiver

- **Récepteur Broadcast, il permet à une application d'être à l'écoute des autres afin de répondre aux objets Intent qui lui sont destinés et qui sont envoyés par d'autres composants applicatifs**
  - **Réagir à des évènements extérieurs à l'Application.**
  - **Réveiller une Application**
  - **Réagir à des évènements Système**
  - **Mise hors tension, Batterie faible**
  - **Changement langue du système**
- **Classe `BrodcastReceiver`**

Architecture Logicielle: Fragment

- **Comportement ou une partie de l'interface utilisateur dans une activité.**
- **Possibilité de combiner de multiples fragments en une seule activité**
- **Un fragment doit toujours être intégré dans une activité et le cycle de vie du fragment est directement affectée par le cycle de vie de l'activité du hôte .**



Architecture Logicielle: Intent

- **Une description abstraite d'une opération à effectuer.**
- **Il peut être utilisé avec**
  - startActivity pour lancer une activité,
  - broadcastIntent pour envoyer à tous les composants de BroadcastReceiver intéressées,
  - StartService (intention) ou bindService (intention, ServiceConnection, int) pour communiquer avec un service d'arrière-plan.
- **Messages système qui circulent entre les activités.**

↳ **Exemple : émettre l'intention X lorsque l'utilisateur est à moins de 100 mètres d'un emplacement Y**